

TAAC 2026 Baseline梳理

本代码解读主要以baseline中的train.py为主线来解读整个baseline代码

数据粗览

	user_id	item_id	label_type	label_time	timestamp	user_int_feats_1	user_int_feats_3	user_int_feats_4	user_int_feats_15	user_int_feats_48	...	domain_d_seq_17	domain_d_seq_18	domain
0	3864676	103989760	1	1772725413	1772725140	4	1753.0	6.0	[928, 556, 538, 739, 94]	42.0	...	[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, ...]	[51, 193, 310, 310, 114, 227, 18, 831, 831, 83...]	[0, 0, 143,
1	8507274	106263941	1	1772725312	1772725252	4	1137.0	601.0	[147, 50, 757]	44.0	...	None	None	
2	11951382	44791352	1	1772725742	1772725278	4	1295.0	63.0	[982, 349, 387, 974, 976, 213]	82.0	...	[3, 3, 3, 3, 3, 3, 3, 3, 3, 3, ...]	[256, 899, 33, 256, 899, 882, 256, 200, 732, 6...]	[982, 0, 0, 1411,

本次比赛提供的数据如上这种形式，分散在多个不同的parquet之中，每一行是一个用户的数据，其中主要包含三个特征组：[user_feat组, item_feat组, seq_domain组]，seq_domain又包含abcd四个域，对于单个用户在同个域中的所有数据长度是一致的，比如对于用户a：domain_a_seq_38，domain_a_seq_39，domain_a_seq_40，domain_a_seq_41等a域的所有特征长度都是一致的，可以理解为一个按时间排序的行为序列，每个时间包括了一些side_info，每个domain的形状是(seq_len,side_info_feat),值得一提的是，所有数据的时间排列都是倒叙的，也就是说靠前的数据，离当前越近，越靠后的数据离当前时间越远。

```
DatetimeIndex(['2026-03-05 13:48:00', '2026-03-05 13:47:00',
               '2026-03-05 13:43:00', '2026-03-05 12:36:00',
               '2026-03-05 12:26:00', '2026-03-05 12:24:00',
               '2026-03-05 12:23:00', '2026-03-05 12:22:00',
               '2026-03-05 08:43:00', '2026-03-05 05:22:06',
               ...])
```

代码解读

从train.py的main开始，首先是一些配置，比如模型的存储位置，log_dir这些先不谈

```
args = parse_args()
```

```
# Create output directories.
Path(args.ckpt_dir).mkdir(parents=True, exist_ok=True)
Path(args.log_dir).mkdir(parents=True, exist_ok=True)
Path(args.tf_events_dir).mkdir(parents=True, exist_ok=True)

# Initialize logger and RNG.
set_seed(args.seed)
create_logger(os.path.join(args.log_dir, 'train.log'))
logging.info(f"Args: {vars(args)}")

from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter(args.tf_events_dir)
```

从schem.json的读取开始

```
# ---- Data loading ----
if args.schema_path:
    schema_path = args.schema_path
else:
    schema_path = os.path.join(args.data_dir, 'schema.json')

if not os.path.exists(schema_path):
    raise FileNotFoundError(f"schema file not found at {schema_path}")
```

schema.json是一个特征的配置文件，其样式如下：p1是整体大观，p2是具体内容，比如item_int中包含了item中的全部int特征，有14个，每个特征有三个value，分别对应 特征名称，vocab_size (int型的词表大小) ，dim(该int型的长度大小)

比如下面，item_int的key为0的数据表示，该特征是item_int_feats_5,词表大小309,占据维度1。有的人又要问了，int型的不应该都是1吗，其实不是的，有的特征其实是不定长的，

```
▼ root
  format "raw_parquet"
  ▶ user_int [] 46 items
  ▶ item_int [] 14 items
  ▶ user_dense [] 10 items
  ▶ seq

▼ item_int [] 14 items
  ▼ 0 [] 3 items
    0 5
    1 309
    2 1
  ▶ 1 [] 3 items
```

比如item_int_feats_11: 最长的一个用户的在int11这个特征可能有20个，因此实际上在这个特征中不足20个的是需要填充0至20个的。

```
▼ item_int [] 14 items
  ▶ 0 [] 3 items
  ▶ 1 [] 3 items
  ▶ 2 [] 3 items
  ▶ 3 [] 3 items
  ▶ 4 [] 3 items
  ▶ 5 [] 3 items
  ▼ 6 [] 3 items
    0 11
    1 21528
    2 20
```

```
sample_df.iloc[0]['item_int_feats_11']

array([2980, 2770])
```

总而言之，schema.json表示我们使用的特征名称是什么，以及每个特征的词表大小和维度大小。

过了schema之后是一个seq_max_lens的参数，这个参数的含义是每个域的seq使用多大的长度，没错，在baseline中也并不是将全部序列都用上了，而是只用上了一部分，因为如果使用全部序列是会爆显存的，怎么优化这部分把全部序列用上上分点之一，优化点+1，seq_max_lens的输入是从参数输入的，形式：seq_a:256,seq_b:256,seq_c:512,seq_d:512 定义了四个域使用的序列长度。

```
# Parse per-domain sequence-length overrides.
seq_max_lens = {}
if args.seq_max_lens:
    for pair in args.seq_max_lens.split(','):
        k, v = pair.split(':')
        seq_max_lens[k.strip()] = int(v.strip())
logging.info(f"Seq max_lens override: {seq_max_lens}")
```

数据输入

接下来的train就创建了dataloader，接下来就要进入到dataset的代码里面

```
logging.info("Using Parquet data format (IterableDataset)")
train_loader, valid_loader, pcvr_dataset = get_pcvr_data(
    data_dir=args.data_dir,
    schema_path=schema_path,
    batch_size=args.batch_size,
    valid_ratio=args.valid_ratio,
    train_ratio=args.train_ratio,
    num_workers=args.num_workers,
    buffer_batches=args.buffer_batches,
    seed=args.seed,
    seq_max_lens=seq_max_lens,
)
```

同理，先看get_pcvr_data这个函数：

获取全部的parquet文件path：

```
random.seed(seed)
import glob as _glob
pq_files = sorted(_glob.glob(os.path.join(data_dir, '*.parquet')))
```

获取parquet文件的对应分组，这一点可能大家直接跑baseline也不会注意到，在parquet内部实际上还是有内部分组的。

```
rg_info = []
for f in pq_files:
    pf = pq.ParquetFile(f)
    for i in range(pf.metadata.num_row_groups):
        rg_info.append((f, i, pf.metadata.row_group(i).num_rows))
total_rgs = len(rg_info)
```

大概样子就是如下，rg_info的作用大家后文会看到的，1.用来划分训练集和验证集 2.用来多线程时分到其他线程

0-9代表组号，100表示这个组里面有100个数据

```
[('./data/demo_1000_10rowgroups.parquet', 0, 100),
 ('./data/demo_1000_10rowgroups.parquet', 1, 100),
 ('./data/demo_1000_10rowgroups.parquet', 2, 100),
 ('./data/demo_1000_10rowgroups.parquet', 3, 100),
 ('./data/demo_1000_10rowgroups.parquet', 4, 100),
 ('./data/demo_1000_10rowgroups.parquet', 5, 100),
 ('./data/demo_1000_10rowgroups.parquet', 6, 100),
 ('./data/demo_1000_10rowgroups.parquet', 7, 100),
 ('./data/demo_1000_10rowgroups.parquet', 8, 100),
 ('./data/demo_1000_10rowgroups.parquet', 9, 100)]
```

基于分组数量划分训练集验证集，比如上面有是个分组，当valid_ratio等于0.2时，两个分组的数据会被划分为验证集，8个分组会被作为训练集，并且，训练集就是 0-7的分组，验证集是8、9两个组，就是按照组的顺序来的，不过最神的来了，train_ratio可以控制训练集的比例，比如我用0.8，训练集的实际大小会变成8*0.8，肯定有他的小心思吧。所以这个划分方式就导致大家的验证集是一样的，并且可能比例也没办法和线上对齐，想要很好的本地验证效果的话可以修改这一块。

```

n_valid_rgs = max(1, int(total_rgs * valid_ratio))
n_train_rgs = total_rgs - n_valid_rgs

# train_ratio: use only the first N% of the training Row Groups.
if train_ratio < 1.0:
    n_train_rgs = max(1, int(n_train_rgs * train_ratio))
    logging.info(f"train_ratio={train_ratio}: using {n_train_rgs} train Row Groups")

train_rows = sum(r[2] for r in rg_info[:n_train_rgs])
valid_rows = sum(r[2] for r in rg_info[n_train_rgs:])

```

然后就是进入dataset的类了，其他参数都好说，这里要解释一个参数，clip_vocab,表示是否对超出vocab的数据进行填充，开False就会报错，demo数据里面的确有超过vocab的值，不过训练集中实际不存在超出vocab的值，所有值都是按照某个值的最大值开的。

```

train_dataset = PCVRParquetDataset(
    parquet_path=data_dir,
    schema_path=schema_path,
    batch_size=batch_size,
    seq_max_lens=seq_max_lens,
    shuffle=shuffle_train,
    buffer_batches=buffer_batches,
    row_group_range=(0, n_train_rgs),
    clip_vocab=clip_vocab,
)

```

进入PCVRDataset，首先依旧是读取parquet文件的路径

```

# Accept either a directory or a single file path.
if os.path.isdir(parquet_path):
    import glob
    files = sorted(glob.glob(os.path.join(parquet_path, '*.parquet')))
    print(files)
    if not files:
        raise FileNotFoundError(f"No .parquet files in {parquet_path}")
    self._parquet_files = files
else:
    self._parquet_files = [parquet_path]

```

然后是一些相关配置，oob_stats用于存储超过vocab的数据，rg和上文的统计parquet的组别同理。

```

self.batch_size = batch_size
self.shuffle = shuffle
self.buffer_batches = buffer_batches
self.clip_vocab = clip_vocab
self.is_training = is_training
# Out-of-bound statistics:
# {(group, col_idx): {'count': N, 'max': M, 'min_oob': M, 'vocab': V}}
self._oob_stats = Dict[Tuple[str, int], Dict[str, int]] = {}

# Build the list of Row Groups.
self._rg_list = []
for f in self._parquet_files:
    pf = pq.ParquetFile(f)
    for i in range(pf.metadata.num_row_groups):
        self._rg_list.append((f, i, pf.metadata.row_group(i).num_rows))

if row_group_range is not None:
    start, end = row_group_range
    self._rg_list = self._rg_list[start:end]

```

然后是读取schema的函数：

```

# Load schema.json.
self._load_schema(schema_path, seq_max_lens or {})

```

这里展示user_int作为例子，item_int和user_dense基本是同理

```
def _load_schema(self, schema_path: str, seq_max_lens: Dict[str, int]) -> None:
    """Populate per-group schema information from ``schema_path``."""
    with open(schema_path, 'r', encoding='utf-8') as f:
        raw = json.load(f)

    # ---- user_int: [[fid, vocab_size, dim], ...] ----
    self._user_int_cols: List[List[int]] = raw['user_int']
    self.user_int_schema: FeatureSchema = FeatureSchema()
    self.user_int_vocab_sizes: List[int] = []
    for fid, vs, dim in self._user_int_cols:
        self.user_int_schema.add(fid, dim)
        self.user_int_vocab_sizes.extend([vs] * dim)
```

实际上就是读取前文提到的schema.json，当然实际上这个部分在后面使用的其实不是很多，连带着FeatureSchema类都感觉有些多余，给大家看看，具体存了些什么吧，user_int_cols,一个列表，实际就是上面schema.json里面user_int中的内容，user_int_schema表示每个特征以及其对应的offset，便于把一个user的全部int特征拉通后，再区分不同的特征，user_int_vocab_sizes是长度实际是411，也就是前面提到的假设用户全部int特征拉通后，每个位置对应的embedding大小。

_user_int_cols	user_int_schema	user_int_vocab_sizes
[[1, 6, 1], [3, 1725, 1], [4, 957, 1], [15, 1167, 26], [48, 101, 1], [49, 3, 1], [50, 4, 1],	FeatureSchema(total_dim=411, features=[fid=1: offset=0, length=1 fid=3: offset=1, length=1 fid=4: offset=2, length=1 fid=15: offset=3, length=26 fid=48: offset=29, length=1 fid=49: offset=30, length=1 fid=50: offset=31, length=1	[6, 1725, 957, 1167, 1167, 1167, 1167, ----

比较下来，seq的schema读取会相对而言更复杂一些，但其实也大差不差，可以参照着左边的图片来读右边的代码会更清晰一些

```
▼ seq_a
  prefix "domain_a_seq"
  ts_fid 39
  ▼ features [] 9 items
    ▼ 0 [] 2 items
      0 38
      1 745286
    ► 1 [] 2 items
    ► 2 [] 2 items
    ► 3 [] 2 items
    ► 4 [] 2 items
    ► 5 [] 2 items
    ► 6 [] 2 items
    ► 7 [] 2 items
    ► 8 [] 2 items
```

```
# ---- sequence domains ----
self._seq_cfg: Dict[str, Dict[str, Any]] = raw['seq']
self.seq_domains: List[str] = sorted(self._seq_cfg.keys())
self.seq_feature_ids: Dict[str, List[int]] = {}
self.seq_vocab_sizes: Dict[str, Dict[int, int]] = {}
self.seq_domain_vocab_sizes: Dict[str, List[int]] = {}
self.ts_fids: Dict[str, Optional[int]] = {}
self.sideinfo_fids: Dict[str, List[int]] = {}
self._seq_prefix: Dict[str, str] = {}
self._seq_maxlen: Dict[str, int] = {}

for domain in self.seq_domains:
    cfg = self._seq_cfg[domain]
    self._seq_prefix[domain] = cfg['prefix']
    ts_fid = cfg['ts_fid']
    self.ts_fids[domain] = ts_fid

    all_fids = [fid for fid, vs in cfg['features']]
    self.seq_feature_ids[domain] = all_fids
    self.seq_vocab_sizes[domain] = {fid: vs for fid, vs in cfg['features']}

    sideinfo = [fid for fid in all_fids if fid != ts_fid]
    self.sideinfo_fids[domain] = sideinfo
    self.seq_domain_vocab_sizes[domain] = [
        self.seq_vocab_sizes[domain][fid] for fid in sideinfo
    ]

# maxlen: from seq_max_lens arg; unspecified domains fall back to 256.
self._seq_maxlen[domain] = seq_max_lens.get(domain, 256)
```

在load完schema之后之后，dataset就要进入batch的计算了，没错，batch实际上是在dataset里面就计算出来了，而不是放在dataloader之中，计算batch之前，下面这个代码统计了一下parquet中的列名，以及列号。

```
# ---- Pre-compute column index lookup ----
pf = pq.ParquetFile(self._parquet_files[0])
schema_names = pf.schema_arrow.names
self._col_idx = {name: i for i, name in enumerate(schema_names)}
```

打印一下col_idx如下,dict文件key是列名, value是列号, 后面将会多次用到:

```
train_dataset._col_idx
```

```
{'user_id': 0,
 'item_id': 1,
 'label_type': 2,
 'label_time': 3,
 'timestamp': 4,
 'user_int_feats_1': 5,
 'user_int_feats_3': 6,
 'user_int_feats_4': 7,
 'user_int_feats_15': 8,
 'user_int_feats_48': 9,
 'user_int_feats_49': 10,
```

然后是创建buf, 也就是缓存, 而且他的这个创建方式还有一个好处在于, 所有特征大小都被提前计算好了, 所以对于缺失值基本不需要再额外进行填充, 省下来了一点时间。total_dim就是所有的int型长度, 序列的特征是, _buf_seq 它的形状是(B,n_feats,maxlen),可以看一下下面的代码, buf_seq_tb是时间分桶

```
# ---- Pre-allocate numpy buffers ----
B = batch_size
self._buf_user_int = np.zeros((B, self.user_int_schema.total_dim), dtype=np.int64)
self._buf_item_int = np.zeros((B, self.item_int_schema.total_dim), dtype=np.int64)
self._buf_user_dense = np.zeros((B, self.user_dense_schema.total_dim), dtype=np.float32)
self._buf_seq = {}
self._buf_seq_tb = {}
self._buf_seq_lens = {}
for domain in self.seq_domains:
    max_len = self._seq_maxlen[domain]
    n_feats = len(self.sideinfo_fids[domain])
    self._buf_seq[domain] = np.zeros((B, n_feats, max_len), dtype=np.int64)
    self._buf_seq_tb[domain] = np.zeros((B, max_len), dtype=np.int64)
    self._buf_seq_lens[domain] = np.zeros(B, dtype=np.int64)
```

依旧是预计算特征, 这个user_int_plan才是后面读入特征的时候真正会使用的, 也就是有用的部分, 这就是为什么上面说到FeatureSchema会有点多余的感觉, 因为在后续dataset中作用根本不大, 也就是说, 目前dataset还有简化的空间。

```
# ---- Pre-compute (col_idx, offset, vocab_size) plans for int columns ----
self._user_int_plan = [] # [(col_idx, dim, offset, vocab_size), ...]
offset = 0
for fid, vs, dim in self._user_int_cols:
    ci = self._col_idx.get(f'user_int_feats_{fid}')
    self._user_int_plan.append((ci, dim, offset, vs))
    offset += dim
```


序列特征依旧同理，大家需要知道的就是，对于int型特征我们需要进行拉通，然后记录每个特征的ci是为了从parquet找到这个特征，记录offset是为了int拉通的时候找到特征该放的位置，记录每个int型特征的vocab_size去便于后续创建embedding，这样做都是为了更方便。四个plan都计划出来之后，dataset的初始化就完成了

```
# Sequence column plan: {domain: [(col_idx, feat_slot, vocab_size), ...], ts_col_idx}
self._seq_plan = {}
for domain in self.seq_domains:
    prefix = self._seq_prefix[domain]
    sideinfo_fids = self.sideinfo_fids[domain]
    ts_fid = self.ts_fids[domain]
    side_plan = []
    for slot, fid in enumerate(sideinfo_fids):
        ci = self._col_idx.get(f'{prefix}_{fid}')
        vs = self.seq_vocab_sizes[domain][fid]
        side_plan.append((ci, slot, vs))
    ts_ci = self._col_idx.get(f'{prefix}_{ts_fid}') if ts_fid is not None else None
    self._seq_plan[domain] = (side_plan, ts_ci)
```

接下来来看dataset的迭代过程，主要是在iter这个函数里面，一般的会采用getitem的方式来做数据的迭代，但是对于parquet数据那样太慢了，所以直接用iter一次读一个batch的方式会加速很多，前面是多线程加速相关的，主包对这一块没有那么了解，不过rg_list又一次出现了，总之意思就是就是把不同的rg放到不同的worker，pf是读取的parquet，用iter_batches表示一次取出1个batch的数据，假设bs是256，一个batch就是pq数据中的256行，convert_batch函数的作用就是这256行数据变成一个dict of tensor可以输入模型的形式，然后这里还会做一个batch缓存的一个小组，buffer_batches用于提前缓存n个batch，这部分也是多线程相关的，主包没有那么了解，大家知道，大致的作用就行了。

```
def __iter__(self) -> Iterator[Dict[str, Any]]:
    worker_info = torch.utils.data.get_worker_info()
    rg_list = self._rg_list
    if worker_info is not None and worker_info.num_workers > 1:
        rg_list = [rg for i, rg in enumerate(rg_list)
                    if i % worker_info.num_workers == worker_info.id]

    buffer: List[Dict[str, Any]] = []
    for file_path, rg_idx, _ in rg_list:
        pf = pq.ParquetFile(file_path)
        for batch in pf.iter_batches(batch_size=self.batch_size, row_groups=[rg_idx]):
            batch_dict = self._convert_batch(batch)
            if self.shuffle and self.buffer_batches > 1:
                buffer.append(batch_dict)
                if len(buffer) >= self.buffer_batches:
                    yield from self._flush_buffer(buffer)
                    buffer = []
            else:
                yield batch_dict

    if buffer:
        yield from self._flush_buffer(buffer)

    del buffer
    gc.collect()
```

接下来就是convert_batch函数，前期提要，batch是parquet中bs大行的数据，可以通过batch.column(列号)获取某一列的全部数据，self._col_idx[column_name]返回的是对应列名的列号

```
# ---- meta ----
timestamps = batch.column(self._col_idx['timestamp']).to_numpy().astype(np.int64)
if self.is_training:
    labels = (batch.column(self._col_idx['label_type']).fill_null(0)
              .to_numpy(zero_copy_only=False).astype(np.int64) == 2).astype(np.int64)
else:
    labels = np.zeros(B, dtype=np.int64)
user_ids = batch.column(self._col_idx['user_id']).to_pylist()
```

讲解数据的获取例子还是从两个，一个是user_int,一个是序列数据。user_int的数据获取如下：

前期提要，buf_user_int是一个 (B,total_dim) 形状的全零numpy，col依旧是获取某个user_int列的一个batch的数据，对于dim=1，也就是单int型的数据（category类型）是填充缺失值为0后转为numpy，还有将小于0的数值全部用0转换，之后会进行一个oob的记录，在self._record_oob这个函数里面（右图），会找出数据中大于vocab的数据并记录超出的最大值最小值，然后将超出的置为0，不过训练集中不存在超出的部分，但是如果大家对这部分做一下数据分析就能发现一些小彩蛋，算是一个提升点之一。user_int[:,offset]表示将offset位置的数据替换为这一列的数据。

```
user_int = self._buf_user_int[:B]
user_int[:] = 0
for ci, dim, offset, vs in self._user_int_plan:
    col = batch.column(ci)
    if dim == 1:
        arr = col.fill_null(0).to_numpy(zero_copy_only=False).astype(np.int64)
        arr[arr <= 0] = 0
        if vs > 0:
            self._record_oob('user_int', ci, arr, vs)
        else:
            arr[:] = 0
            user_int[:, offset] = arr
    else:
        padded, _ = self._pad_varlen_int_column(col, dim, B)
        if vs > 0:
            self._record_oob('user_int', ci, padded, vs)
        else:
            padded[:] = 0
            user_int[:, offset:offset + dim] = padded

def _record_oob(
    self,
    group: str,
    col_idx: int,
    arr: "npt.NDArray[np.int64]",
    vocab_size: int,
) -> None:
    """Record out-of-bound indices and (optionally) clip them to 0,
    without printing to the console.
    """
    oob_mask = arr >= vocab_size
    if not oob_mask.any():
        return
    key = (group, col_idx)
    oob_vals = arr[oob_mask]
    n = int(oob_mask.sum())
    mx = int(oob_vals.max())
    mn = int(oob_vals.min())
    if key in self._oob_stats:
        s = self._oob_stats[key]
        s['count'] += n
        s['max'] = max(s['max'], mx)
        s['min_oob'] = min(s['min_oob'], mn)
    else:
        self._oob_stats[key] = {
            'count': n, 'max': mx, 'min_oob': mn, 'vocab': vocab_size,
        }
    if self.clip_vocab:
        arr[oob_mask] = 0
    else:
        raise ValueError(
            f"{group} col_idx={col_idx}: {n} values out of range "
            f"[0, {vocab_size}], actual=[{mn}, {mx}]. "
            f"Use clip_vocab=True to clip or fix schema.json")
```

对于dim>1的数据列，这可认为是一个array int类型的数据，对于这部分数据填补缺失值的办法，代码中使用了，_pad_varlen_int_column函数，假设arrow_col形状是(B,feat_num)，这个feat_num是不定长的，表示了B个用户这个特征的list，使用offset统计每个用户的偏移量，长度是用户数，假设第一个用户特征[1,2,3]，第二个用户特征是[1,2,3,4,5]，那么offset实际为[0,2]，value则将所有list拉通为一个list，比如刚刚两个用户会变成[1,2,3,1,2,3,4,5]

offset里面的偏移量表示，[0:3]的数据属于第一个用户，[3:]的数据属于第二个用户，了解了这一点之后就很容易看懂baseline里的填充流程，padded先是开到最大，假设max_len是5，那么padded就是[[0,0,0,0,0],[0,0,0,0,0]]，这是两个用户该特征的最大长度，然后进行遍历每一个用户，对于用户1，他的数据是0:3，那么就填充为[1,2,3,0,0]，其他的所有填充也是同理，float的填充也是同理，后续不赘述。


```
def _pad_varlen_int_column(
    self,
    arrow_col: "pa.ListArray",
    max_len: int,
    B: int,
) -> Tuple["npt.NDArray[np.int64]", "npt.NDArray[np.int64]"]:
    """Pad an Arrow `ListArray` of ints to shape `[B, max_len]`."""

    Values <= 0 are mapped to 0 (padding). Note: the raw data contains -1
    (missing); currently treated the same way as 0 (padding).

    Returns:
        A tuple `(padded, lengths)` where `padded` has shape
        `[B, max_len]` and `lengths` has shape `[B]`.
    """
    offsets = arrow_col.offsets.to_numpy()
    values = arrow_col.values.to_numpy()

    padded = np.zeros((B, max_len), dtype=np.int64)
    lengths = np.zeros(B, dtype=np.int64)

    for i in range(B):
        start, end = int(offsets[i]), int(offsets[i + 1])
        raw_len = end - start
        if raw_len <= 0:
            continue
        use_len = min(raw_len, max_len)
        padded[i, :use_len] = values[start:start + use_len]
        lengths[i] = use_len

    padded[padded <= 0] = 0
    return padded, lengths
```

```
def _pad_varlen_float_column(self, arrow_col, max_dim, B) :
    offsets = arrow_col.offsets.to_numpy()
    values = arrow_col.values.to_numpy()

    padded = np.zeros((B, max_dim), dtype=np.float32)
    for i in range(B):
        start, end = int(offsets[i]), int(offsets[i + 1])
        raw_len = end - start
        if raw_len <= 0:
            continue
        use_len = min(raw_len, max_dim)
        padded[i, :use_len] = values[start:start + use_len]

    return padded
```

对于每个域的序列数据处理起来会更复杂一些，但是处理int和float的核心思路还是和上面一样，对于单个序列，首先取出他的side_info和时间戳的列，时间戳的列后续要用做time_bucket分桶，前情提要，out是某个序列的全部特征，shape为（B, n_feat,max_seq_len）如果单看某一个时间，其实做法就和获取一个用户特征的方法是一样的，

但是这里使用的方法更类似于上文提到的int array类型的填充，就是把某一个int当成是max_seq_len dim大小的int array去做偏移和value就好了。time_bucket分桶部分，是用当前的时间戳减去某个域的时间戳，从而计算出序列中的每一个时间与当前的差值，然后根据划定的时间差进行分桶

```
# ---- Sequence features: fused padding directly into the 3D buffer ----
for domain in self.seq_domains:
    max_len = self._seq_maxlen[domain]
    side_plan, ts_ci = self._seq_plan[domain]

    # Write directly into the pre-allocated 3D buffer.
    out = self._buf_seq[domain][:B]
    out[:] = 0
    lengths = self._buf_seq_lens[domain][:B]
    lengths[:] = 0

    # Fused path: first collect (offsets, values, vocab_size, col_idx)
    # for every side-info column, then fill the buffer in a single pass.
    col_data = []
    for ci, slot, vs in side_plan:
        col = batch.column(ci)
        col_data.append((col.offsets.to_numpy(), col.values.to_numpy(), vs, ci))

    for c, (offs, vals, vs, ci) in enumerate(col_data):
        for i in range(B):
            s = int(offs[i])
            e = int(offs[i + 1])
            rl = e - s
            if rl <= 0:
                continue
            ul = min(rl, max_len)
            out[i, c, :ul] = vals[s:s + ul]
            if ul > lengths[i]:
                lengths[i] = ul

    # Values <= 0 -> 0.
    out[out <= 0] = 0
```

```
# Time bucketing.
time_bucket = self._buf_seq_tb[domain][:B]
time_bucket[:] = 0
if ts_ci is not None:
    ts_col = batch.column(ts_ci)
    ts_offs = ts_col.offsets.to_numpy()
    ts_vals = ts_col.values.to_numpy()
    # Pad timestamps into shape (B, max_len).
    ts_padded = np.zeros((B, max_len), dtype=np.int64)
    for i in range(B):
        s = int(ts_offs[i])
        e = int(ts_offs[i + 1])
        rl = e - s
        if rl <= 0:
            continue
        ul = min(rl, max_len)
        ts_padded[i, :ul] = ts_vals[s:s + ul]

    ts_expanded = timestamps.reshape(-1, 1)
    time_diff = np.maximum(ts_expanded - ts_padded, 0)
    raw_buckets = np.clip(
        np.searchsorted(BUCKET_BOUNDARIES, time_diff.ravel()),
        0, len(BUCKET_BOUNDARIES) - 1,
    )
    buckets = raw_buckets.reshape(B, max_len) + 1
    buckets[ts_padded == 0] = 0
    time_bucket[:] = buckets

result[f'{domain}_time_bucket'] = torch.from_numpy(time_bucket.copy())
```

接下来展示一下返回的结果，dataset的部分到这里就结束了！

[illegible]

```
train_loader = DataLoader(
    train_dataset, batch_size=None,
    num_workers=num_workers, pin_memory=use_cuda, **_train_kw,
)

valid_dataset = PCVRParquetDataset(
    parquet_path=data_dir,
    schema_path=schema_path,
    batch_size=batch_size,
    seq_max_lens=seq_max_lens,
    shuffle=False,
    buffer_batches=0,
    row_group_range=(n_train_rgs, total_rgs),
    clip_vocab=clip_vocab,
)

valid_loader = DataLoader(
    valid_dataset, batch_size=None,
    num_workers=0, pin_memory=use_cuda,
)

logging.info(f"Parquet train: {train_rows} rows, valid: {valid_rows} rows, "
            f"batch_size={batch_size}, buffer_batches={buffer_batches}")

return train_loader, valid_loader, train_dataset
```

```

# -- NS groups -----
if args.ns_groups_json and os.path.exists(args.ns_groups_json):
    logging.info(f"Loading NS groups from {args.ns_groups_json}")
    with open(args.ns_groups_json, 'r') as f:
        ns_groups_cfg = json.load(f)
        user_fid_to_idx = {fid: i for i, (fid, _) in enumerate(pcvr_dataset.user_int_schema.entries)}
        item_fid_to_idx = {fid: i for i, (fid, _) in enumerate(pcvr_dataset.item_int_schema.entries)}
        user_ns_groups = [user_fid_to_idx[f] for f in fids] for fids in ns_groups_cfg['user_ns_groups'].values()]
        item_ns_groups = [item_fid_to_idx[f] for f in fids] for fids in ns_groups_cfg['item_ns_groups'].values()]
        logging.info(f"User NS groups ({len(user_ns_groups)}): {list(ns_groups_cfg['user_ns_groups'].keys())}")
        logging.info(f"Item NS groups ({len(item_ns_groups)}): {list(ns_groups_cfg['item_ns_groups'].keys())}")
else:
    logging.info("No NS groups JSON found, using default: each feature as one group")
    user_ns_groups = [[i] for i in range(len(pcvr_dataset.user_int_schema.entries))]
    item_ns_groups = [[i] for i in range(len(pcvr_dataset.item_int_schema.entries))]

"user_ns_groups": {
    "U1": [1, 15],
    "U2": [48, 49, 89, 90, 91],
    "U3": [80],
    "U4": [51, 52, 53, 54, 86],
    "U5": [82, 92, 93],
    "U6": [50, 60, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109],
    "U7": [3, 4, 55, 56, 57, 58, 59, 62, 63, 64, 65, 66]
},

"item_ns_groups": {
    "I1": [13, 13],
    "I2": [5, 6, 7, 8, 12],
    "I3": [16, 81, 83, 84, 85],
    "I4": [9, 10]
}
}

```

一个就够了，这个spec的长度等于特征数量，是按照拉通特征的顺序排的，这个主要用于后续模型的初始化。

```
# ---- Build model ----
user_int_feature_specs = build_feature_specs(
    pcvr_dataset.user_int_schema, pcvr_dataset.user_int_vocab_sizes
)
item_int_feature_specs = build_feature_specs(
    pcvr_dataset.item_int_schema, pcvr_dataset.item_int_vocab_sizes

def build_feature_specs(
    schema: FeatureSchema,
    per_position_vocab_sizes: List[int],
) -> List[Tuple[int, int, int]]:
    """Build feature_specs of the form [(vocab_size, offset, length), ...]
    ordered by the positions recorded in 'schema.entries'."""
    specs: List[Tuple[int, int, int]] = []
    for fid, offset, length in schema.entries:
        vs = max(per_position_vocab_sizes[offset:offset + length])
        specs.append((vs, offset, length))
    return specs
```

然后就是模型的初始化部分了，这里的参数极多，baseline确实是写的很完备这一次，几乎没有去年的低级错误了，参数就先不一个个的意义介绍，后续的model代码里面出现了就会提一下，大多数的参数大家也知道含义。

```
model_args = {
    "user_int_feature_specs": user_int_feature_specs,
    "item_int_feature_specs": item_int_feature_specs,
    "user_dense_dim": pcvr_dataset.user_dense_schema.total_dim,
    "item_dense_dim": pcvr_dataset.item_dense_schema.total_dim,
    "seq_vocab_sizes": pcvr_dataset.seq_domain_vocab_sizes,
    "user_ns_groups": user_ns_groups,
    "item_ns_groups": item_ns_groups,
    "d_model": args.d_model,
    "emb_dim": args.emb_dim,
    "num_queries": args.num_queries,
    "num_hyformer_blocks": args.num_hyformer_blocks,
    "num_heads": args.num_heads,
    "seq_encoder_type": args.seq_encoder_type,
    "hidden_mult": args.hidden_mult,
    "dropout_rate": args.dropout_rate,
    "seq_top_k": args.seq_top_k,
    "seq_causal": args.seq_causal,
    "action_num": args.action_num,
    "num_time_buckets": NUM_TIME_BUCKETS if args.use_time_buckets else 0,
    "rank_mixer_mode": args.rank_mixer_mode,
    "use_rope": args.use_rope,
    "rope_base": args.rope_base,
    "emb_skip_threshold": args.emb_skip_threshold,
    "seq_id_threshold": args.seq_id_threshold,
    "ns_tokenizer_type": args.ns_tokenizer_type,
    "user_ns_tokens": args.user_ns_tokens,
    "item_ns_tokens": args.item_ns_tokens,
}
```

```
model = PCVRHyFormer(**model_args).to(args.device)
```

接下来就是一千多行的模型部分了，真的头大。

